Roman Burdakov, Abdul Rehman, Lindsay Yu, Daniel Menard, Vipin Myneni

15.773 Hands-on Deep Learning - Spring 2026

Final Project Report

Predicting U.S. Domestic Opening Weekend from Pre-Release Digital Signals

Report scope. This report documents the final frozen notebook for the team's opening-weekend forecasting project. It emphasizes the official primary benchmark, the final retained model lineup, the role of feature engineering, and the main experimental lessons. Earlier exploratory branches - including Ridge-style baselines, an older scalar-tab Transformer, and additional benchmark filters - are discussed where they materially informed the final conclusions, but they were not retained as headline results in the final submission.

Data sources used: Box Office Mojo scraping, TMDB API, and YouTube Data API v3.

Final notebook and data: Google Colab and Google Drive (data to run notebook)

Summary

The goal of this project was to predict U.S. domestic opening weekend box office using only information that is plausibly available before release. The team deliberately moved away from relying on final theater count in the main configuration, because that variable is both highly predictive and partly downstream of studio expectations. The resulting problem is harder, but more realistic for an early forecasting use case.

The final frozen notebook uses 344 labeled films on a random 75/25 train-test split, with movie metadata, engineered title and franchise proxies, trailer and video metadata, transfer-learned text embeddings, and YouTube comment-derived statistics and embeddings. A central conclusion of the project is that data quality and feature quality mattered more than most late-stage architecture tweaks. Better trailer candidate matching, better comment quality, stronger title/franchise indicators, and richer comment-bin features improved nearly every model family.

The best final model is a ResNet-style multimodal neural network that combines a 93-feature tabular branch with a compact 34-dimensional reduced text block. In the frozen notebook it achieves test R^2_log = 0.586, test MAE_log = 0.489, test R^2 in dollar space = 0.549, and test MAE of about $18.8M. The strongest non-neural benchmark is XGBoost with tabular and text inputs, which is very competitive in log space (test R^2_log = 0.577) but substantially weaker in dollar-space calibration. The best Transformer retained in the final comparison is a late-fusion design with a text Transformer branch plus a tabular MLP

branch; it improves materially over the older scalar-tab Transformer family but remains below both ResNet and XGBoost.

The project's most important lesson is that the remaining errors are not random. The hardest failures are concentrated in blockbuster, franchise, family, remake, and event-like titles whose online pre-release signal is sparse, misleading, or structurally different from normal releases. That means the path forward is less about another broad hyperparameter sweep and more about regime-aware features, better eventness/franchise priors, and stronger calibration for top-end movies.

1. Project Objective and Benchmark Definition

The project focused on forecasting domestic opening weekend box office before release using pre-release metadata, trailer-related information, and early audience reaction signals from YouTube. We treated this as a supervised regression problem, with the goal of predicting a continuous dollar outcome rather than relying only on descriptive comparisons or simple benchmark heuristics.

One modeling choice that shaped the project was how to handle opening theater count. Rather than using it as a standard input in the primary benchmark, we left it behind a feature flag and excluded it from the main reported setup. In early, weaker model versions, theater count tended to dominate the prediction and mask whether the model was actually learning from the richer metadata, text, and social-signal features. In later versions, including theater count did improve performance, but the gains were not dramatic enough to change the overall conclusions. For that reason, the main benchmark was kept in the stricter no-theater configuration, while results with theater count enabled are reported separately in Appendix 1.

We also considered reframing the task as a classification problem, predicting box office ranges rather than an exact revenue value. In practice, that version of the problem was materially easier, especially when the number of classes was chosen appropriately. However, we ultimately decided to keep the final project centered on regression because predicting the actual opening weekend gross is the more demanding and more informative problem. That choice made the task harder, but it also provided a stronger test of whether pre-release digital and metadata signals could support meaningful forecasting accuracy.

The official primary benchmark in the frozen notebook is a random train-test split with 344 movies released in the US between 2016 and 2026, yielding 258 training examples and 86 test examples. The main comparison is conducted in log space using R^2_log and MAE_log, with dollar-space metrics retained as a secondary calibration check. This choice reflects the long-tailed nature of box office outcomes: the difference between a $5M opener and a $15M opener matters, but so does the difference between a $50M opener and a $150M opener, and log-space evaluation better stabilizes those regimes.

2. Data Collection and Dataset Construction

Data was assembled from three primary sources: Box Office Mojo for opening-weekend labels, the TMDB API for movie metadata, and the YouTube Data API plus downloaded JSON/video-comment archives for trailer-level and comment-level inputs. The final notebook loads 344 movies with labels and matched YouTube identifiers. It also audits the local YouTube archive, finding complete video and comment file coverage for the matched IDs used in the final modeling dataset.

The data pipeline evolved substantially during the project. An important early issue was trailer candidate quality: a simple search for a movie title plus the word trailer could attach the wrong video or a suboptimal upload, which polluted both video metadata and comment collections. The team therefore revisited candidate selection, adding rules around official channels, trailer-title patterns, and semantic text quality. This was one of the most consequential improvements in the entire project, because nearly all downstream models benefited once trailer matching became more reliable.

YouTube comments were also treated more carefully over time. The notebook records not only pooled comment text and counts, but also bin-level statistics that capture how audience reaction evolves over portions of the campaign. In the final pipeline, 309 titles had directly parsed usable comment rows, while the full 344-title modeling frame was preserved by using explicit masks and missingness-aware handling rather than dropping movies wholesale. This decision preserved sample size while still distinguishing between genuine audience signal and no-signal cases.

3. Feature Engineering

The final feature set combines four broad classes of information. First, the model uses structured metadata and engineered tabular variables: release timing, runtime, language, distributor indicators, genre dummies, sequel/title markers, family-like and four-quadrant proxies, and calendar-corridor variables such as Thanksgiving or Christmas release windows. Second, it uses safe trailer/video features such as duration, tag count, title patterns (official, teaser, final), and channel-quality indicators.

Third, the project uses transfer learning to construct dense semantic representations. The frozen notebook uses a SentenceTransformer model (all-mpnet-base-v2) to generate 768-dimensional embeddings for movie description text, trailer/video text, pooled comments, and comment bins. These channels are not simply concatenated raw. Instead, the notebook evaluates compact dimensionality reduction and settles on a reduced text block that is deliberately small enough to avoid overfitting. The final reduced representation contains 34 features: 8 principal components each for description and video text, 4 for pooled comments, 2 for each of four comment bins, and 6 binary mask indicators for text availability.

The PCA choice mattered. Earlier experiments compared different compression levels, and the final notebook effectively centers on eight dimensions as the strongest trade-off for the major dense text channels. That choice preserved enough semantic signal to be useful while avoiding the instability that often follows from feeding very high-dimensional embeddings into small-sample regression models.

Fourth, the project adds a rich set of comment-derived numeric features. These include comment count and share by time bins, sentiment means and positive-share measures by bin, comment diversity, like concentration, punctuation/emphasis measures, and trend features. Exploratory plots in the notebook show that average YouTube sentiment is mildly positive on average and only weakly related to opening weekend by itself. The key lesson was not that sentiment is useless; rather, sentiment is a weak standalone predictor but a useful supplementary feature when combined with comment volume, metadata, title/franchise cues, and broader engagement patterns.

4. Modeling Strategy and Experimental Evolution

The team did not arrive at the final lineup immediately. The early baseline path started with simpler tabular-plus-sentiment networks and traditional linear methods. Ridge and Elastic Net baselines were useful reference points because they were competitive enough to keep the team honest about whether more complex architectures were actually earning their extra complexity. Those linear experiments were eventually removed from the final submission table to keep the report focused, but they served an important role: they showed that the feature set already contained real predictive signal before the strongest neural models were finalized.

The project then moved toward multimodal neural architectures. A deliberately small text-only MLP was retained as a sanity-check baseline. This model uses the reduced text block alone, trained as a five-seed ensemble with a small hidden layer and model-specific callback policy. It is not the final winner, but it demonstrates that the text pipeline alone contains meaningful pre-release signal, even before adding structured metadata and engineered social features.

The strongest neural direction became a ResNet-style tab+text model. In the frozen notebook, the architecture takes a 93-feature tabular branch and a 34-feature reduced text branch, normalizes them separately, projects both into compact dense layers, fuses them, and then passes the fused representation through a small stack of residual blocks. The winning configuration uses width 120, two residual blocks, dropout 0.15, L2 regularization 1.5e-3, learning rate 1.6e-4, batch size 8, and no Gaussian input noise. Across the experiments, this family showed the best balance of signal extraction, optimization stability, and interpretable tuning behavior.

Transformer experiments followed two distinct phases. The older phase attempted to represent scalar/tabular inputs as Transformer-style tokens. Those models improved somewhat over time, but they remained sensitive to tokenization decisions, depth, batch

size, and feed-forward width, and ultimately underperformed the better ResNet variants. Because that older scalar-tab Transformer family was no longer competitive and added complexity without changing the final conclusions, it was removed from the final focused submission.

The later Transformer phase adopted a better design: a late-fusion architecture in which description, video, pooled comments, and comment-bin embeddings are processed through a Transformer text branch, while a compact 31-feature tabular set is processed through a separate MLP branch and fused late. This was a genuine architectural improvement. A stage-1 screening search over five targeted candidates was followed by a stage-2 multi-seed refit of the two best settings, and the final chosen configuration used token dimension 32, two heads, two Transformer blocks, FFN width 128, dropout 0.10, L2 = 1e-3, learning rate 8e-5, batch size 20, and a slightly enlarged tabular branch (96 then 48 hidden units).

Hyperparameter tuning throughout the project followed a pragmatic two-stage design. Stage 1 was a fast scan on a fixed validation split to eliminate clearly weak candidates. Stage 2 refit the top configurations across multiple random seeds to identify robust winners. This distinction turned out to matter because some apparent stage-1 winners did not survive multi-seed comparison. The notebook also preserves one further experimental lesson: callback policy should be model-specific. Increasing early-stopping patience globally helped the tiny text-only baseline but hurt both ResNet and Transformer branches, so the final training helpers treat those models differently.

5. Final Results

Table 1 summarizes the final retained model lineup from the frozen notebook. The ranking below reflects the official primary benchmark only; exploratory filtered-sensitivity runs are discussed afterward and are not directly comparable to these headline results.

Model	Role	Test R^2_log	Test MAE_log	Test R^2 ($)	Test MAE ($)
ResNet-style tab+text NN	Winner	0.586	0.489	0.549	$18.79M
XGBoost Tab+Text	Runner-up / best non-neural	0.577	0.495	0.365	$20.36M
Late-fusion Transformer	Best Transformer	0.535	0.532	0.423	$21.50M

Tiny Text-Only MLP	Baseline	0.470	0.540	0.376	$20.56M
XGBoost Text-Only	Secondary baseline	0.405	0.572	0.263	$22.30M

The most important comparison is between the winner and the runner-up. XGBoost with tabular and text inputs is very strong in log space and nearly matches the ResNet on the primary ranking metric. That result is important because it shows the engineered feature set is genuinely powerful rather than being useful only inside a neural architecture. At the same time, ResNet remains the better final model because it is slightly stronger in log space and much better calibrated in dollar space. In other words, XGBoost is excellent at ordering movies on a compressed scale, but ResNet handles the largest openings more faithfully.

The late-fusion Transformer is the strongest retained Transformer result and clearly better than the earlier scalar-tab Transformer family. However, it still finishes behind both ResNet and XGBoost Tab+Text. The tiny text-only model also performs better than a trivial baseline would, which confirms that the transfer-learned text pipeline is meaningful on its own, but the multimodal models materially outperform it.

Earlier in the broader project discussions, the team also ran a filtered sensitivity benchmark that removed a small set of especially difficult family/event-like titles with sparse or misleading online signal. On that narrower benchmark, the best ResNet reached roughly test R^2_log 0.631 and test MAE_log 0.455. Those are strong numbers, but they are not the official headline metrics because the benchmark is easier. The right way to interpret them is as evidence that unsupported outliers matter a great deal, not as proof that the official broad benchmark was solved at that level.

6. What Drove Performance and Where Models Still Fail

Permutation importance, mutual information, and XGBoost gain analysis in the frozen notebook point to several repeatedly useful tabular features: runtime, a family/four-quadrant indicator, comment uniqueness ratio, sequel proxy variables, official-trailer title cues, and average early sentiment. Figure 1 shows a representative feature-importance view from the final pipeline. This ranking is instructive because it suggests that the strongest signal does not come from any single glamorous deep-learning component. Instead, the winning models benefit from a layered combination of basic movie metadata, regime-aware engineered features, and coarse but informative audience-response summaries.

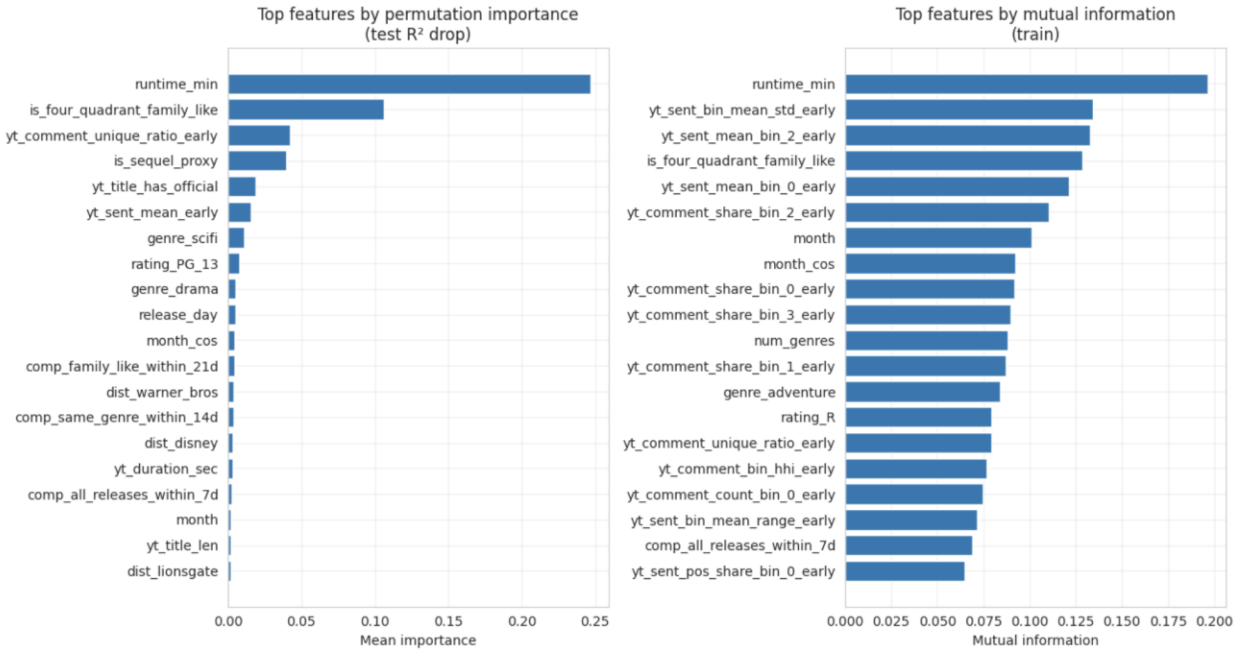

Figure 1: Representative feature-importance view from the final modeling pipeline. The most useful predictors combine movie metadata, title/franchise proxies, and coarse YouTube-derived audience signals, supporting the conclusion that feature quality mattered more than late-stage architecture changes.

The project's error analysis also shows why further generic tuning produced diminishing returns. The worst residuals cluster around specific movie types rather than random noise. Large underpredictions occur on titles such as Joker, Wicked: For Good, Aladdin, and Taylor Swift | The Eras Tour - movies whose opening performance reflects a level of franchise strength, eventness, or cultural saturation that is hard to infer from the available pre-release digital footprint alone. Overpredictions appear on some movies where online discussion was strong or noisy but did not translate into equivalent theatrical demand, such as Ready or Not and Mortal Kombat in the notebook's diagnostics. Figure 2 illustrates this concentration of error in a small number of regime-specific cases.

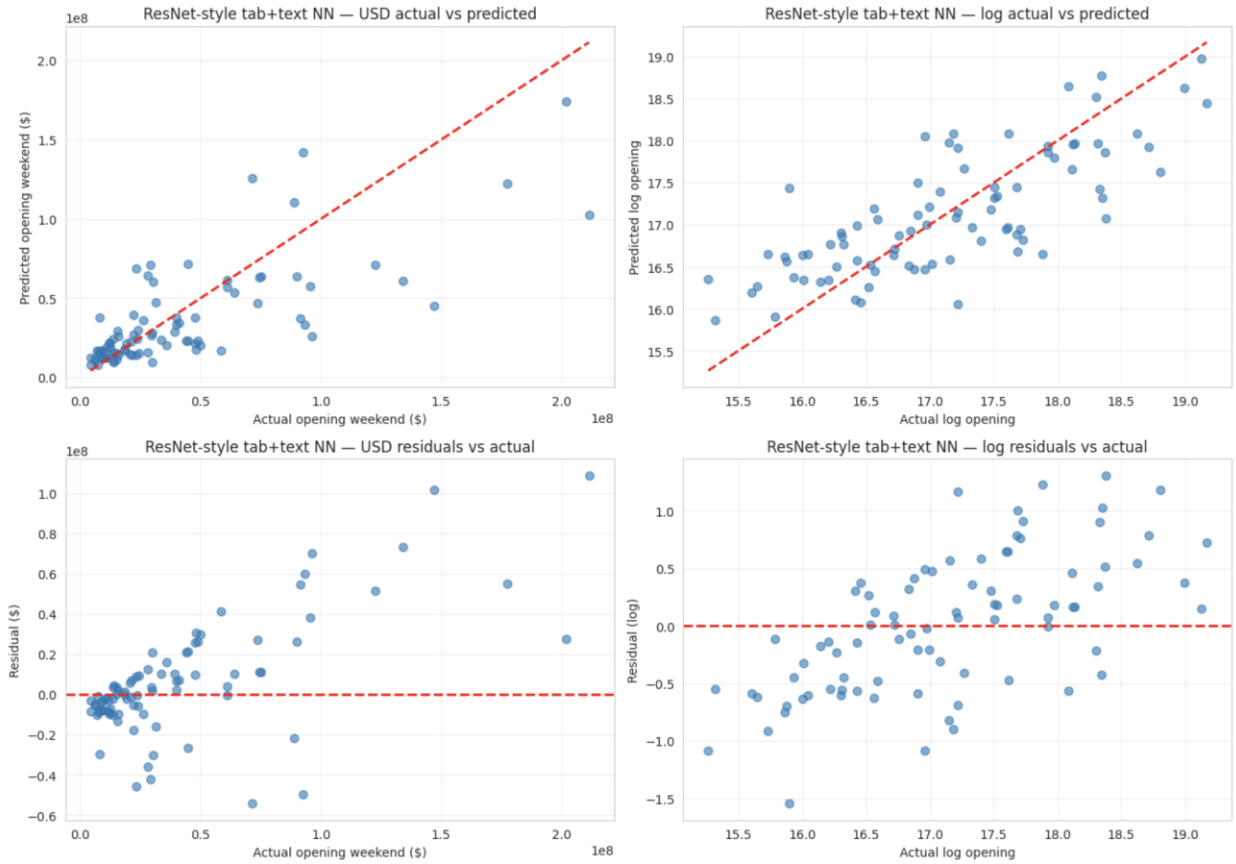

Figure 2. Final-model error diagnostics for the winning ResNet benchmark. The largest residuals are concentrated in blockbuster, franchise, family, remake, and event-like titles, suggesting that the remaining gap is driven more by regime-specific calibration than by generic model capacity.

This pattern changes the interpretation of the remaining performance gap. The bottleneck is no longer simply model capacity. The bottleneck is better regime detection: identifying when a movie belongs to a blockbuster, family, remake, or event-like class whose behavior breaks the usual relationship between pre-release comments and opening weekend. That is also why sentiment should be interpreted cautiously. Exploratory analysis showed that mean sentiment and positive-comment ratio are only weakly related to opening weekend by themselves and exhibit visible discretization and sparsity effects. Their value comes mainly from acting as coarse supplementary indicators inside a wider multimodal system.

7. Experiments Not Retained as Headline Results

Several experiments informed the final modeling decisions but were intentionally omitted from the headline results because they did not change the final recommendation. Ridge and Elastic Net baselines were useful during development because they provided a strong conventional reference point and prevented the team from overclaiming the value of more complex models. However, once the final notebook converged on the strongest retained

lineup, those linear results no longer changed the overall story, so they were not emphasized in the final table.

Similarly, the older scalar-tab Transformer branch was important as a learning step but not as a final recommendation. It demonstrated that Transformers were not automatically superior simply because they were more expressive. In this small-sample setting, representing structured scalars as tokens introduced complexity without enough compensating benefit. The later late-fusion Transformer was a real improvement and is therefore retained; the earlier Transformer family is discussed only to explain why the project moved away from it.

Finally, we also ran a small set of filtered sensitivity benchmarks that produced better performance by excluding a narrow group of unusually difficult titles. These were mainly family-, remake-, and event-like releases, including several high-profile Disney titles, as well as a few cases with sparse or less comparable pre-release social signal. For some Disney releases, official YouTube trailers had comments disabled, while unofficial uploads did not provide equivalent signal quality, which weakened the usefulness of the social features. More broadly, these runs highlighted how strongly a small number of outlier regimes could affect aggregate metrics. We therefore treat the filtered results as a sensitivity analysis rather than as the main benchmark.

8. Lessons Learned, Limitations, and Conclusion

Three lessons stand out most strongly. First, data quality had the highest return on effort. Cleaner joins, better trailer matching, and better comments improved almost every model. Second, multimodal modeling helped, but only when paired with disciplined feature engineering and dimensionality control. The project did not succeed by throwing a large model at raw data; it succeeded by building a compact, trustworthy input representation. Third, the strongest final architecture was not the most exotic one. The ResNet-style tab+text model won because it matched the structure of the problem well and trained reliably on a limited sample.

The main limitations are also clear. The dataset is still small by modern deep-learning standards, the hardest titles are exactly the movies that matter most commercially, and some forms of demand - especially event-like demand - are only partially visible in YouTube activity. The project also relies on one matched trailer/video per title, which is a defensible simplification but not a complete representation of an entire campaign. These limitations help explain why the largest remaining errors are concentrated in specific regimes rather than being evenly distributed.

Overall, the final frozen notebook supports a credible and useful conclusion: it is feasible to predict opening weekend box office before release without relying on theater count as a main feature, and pre-release digital signals materially improve forecasting when they are collected and engineered carefully. The final recommended model family is the ResNet-style multimodal regressor. XGBoost Tab+Text should be presented as the strongest non-

neural benchmark, and the late-fusion Transformer should be presented as the best retained Transformer direction without overstating its competitiveness. The strongest path for future work is not a broad new hyperparameter sweep, but rather richer franchise/event priors, stronger regime-aware features, and better calibration for movies whose online signal underrepresents true opening-weekend demand.

Appendix 1. Results with Opening Theater Count Enabled

As an additional sensitivity check, we reran the same final notebook with **opening theater count enabled** through the feature flag. The purpose of this appendix is to show how much predictive power theater count adds relative to the primary no-theater benchmark.

Model	Test R^2 (log)	Test MAE (log)	Test R^2 ($)	Test MAE ($)
XGB Tab+Text Tuned	**0.7325**	**0.3834**	0.4739	$16,922,358
ResNet-style tab+text NN (full current tab set, 3-seed ensemble)	0.7050	0.4149	**0.5960**	$17,351,166
Tiny Text-Only MLP (5-seed ensemble)	0.4696	0.5402	0.3759	$20,561,230
XGB Text-Only Tuned	0.4050	0.5716	0.2630	$22,303,206
Late-fusion transformer (text transformer + tab MLP, 3-seed ensemble)	-1.2422	1.1469	-2.5530	$51,190,268

Including opening theater count improves the strongest models, but it does not fundamentally change the ranking or the overall conclusions of the project. XGB Tab+Text becomes the best model in log-space metrics, while the ResNet-style multimodal neural network remains strongest in dollar-space (R^2), which suggests that theater count helps both approaches but does not overturn the broader finding that multimodal metadata and social-signal features already carry substantial predictive power on their own. The late-fusion Transformer rerun in this theater-enabled setting was unstable and is shown for completeness rather than as a competitive retained result.

This appendix also helps explain why theater count was not used in the primary benchmark. In earlier, weaker model versions, theater count tended to dominate the prediction and made it harder to tell whether the models were truly learning from richer pre-release metadata, text, and YouTube-based audience features. By the time the final notebook stabilized, enabling theater count did improve performance, but the gains were not large enough to change the core project takeaway: the model can achieve meaningful pre-release opening-weekend prediction accuracy even without relying on theater count as a standard input.